Python – Advanced Python Programming (Module 3)
Complete Interview Questions and Answers

1. What does the open() function do in Python file handling?
Answer: open() is used to open a file for reading, writing, appending, or other operations. It returns a file object.

2. Explain the difference between the file modes 'r', 'w', and 'a' when opening a file in Python.
Answer: • r = Read mode (file must exist) • w = Write mode (creates file or overwrites existing content) • a = Append mode (adds data to end of file)

3. How would you write the string 'Hello, World!' to a new text file named 'greeting.txt' in Python?
```
with open("greeting.txt", "w") as file:
    file.write("Hello, World!")
```

4. What is the purpose of the tell() method in file handling?
Answer: tell() returns the current position of the file pointer.

5. How can you move the file pointer to the beginning of a file after reading some lines?
Answer: Use seek(0).

6. Write a code snippet to read a file line-by-line and print each line.
```
with open("sample.txt","r") as file:
    for line in file:
        print(line.strip())
```

7. What is exception handling in Python, and why is it important?
Answer: Exception handling manages runtime errors and prevents program crashes using try-except blocks.

8. How do you handle multiple exceptions in a single try-except block?
```
try:
    x = int(input())
except (ValueError, TypeError) as e:
    print(e)
```

9. Write a try-except block that catches a ZeroDivisionError and prints 'Cannot divide by zero.'
```
try:
    result = 10/0
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

10. How do you create a custom exception in Python?
```
class MyError(Exception):
    pass
```

11. What does the 'raise' keyword do in Python exception handling?
Answer: raise is used to manually trigger an exception.

12. If you see a long traceback after running your code, how can ChatGPT help you understand and fix the error?
Answer: Paste the traceback into ChatGPT. It can explain the error, identify the faulty line, suggest

fixes, and provide corrected code.

13. What is a class in Python, and how is it different from an object?
Answer: A class is a blueprint. An object is an instance of a class.

14. Explain the purpose of the __init__ method in a Python class.
Answer: __init__() is a constructor used to initialize object attributes when an object is created.

15. Write a simple Python class named 'Car' with attributes 'brand' and 'year'. Create an object of this class.

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

car1 = Car("Toyota", 2024)
print(car1.brand, car1.year)
```

16. What is single inheritance in Python? Give an example scenario where you might use it.
Answer: One child class inherits from one parent class. Example: Employee inherits Person.

17. How does multiple inheritance differ from multilevel inheritance in Python?
Answer: • Multiple: One child inherits from multiple parents. • Multilevel: Inheritance chain Parent → Child → Grandchild.

18. Given a 'Grandparent', 'Parent', and 'Child' class, how would you implement multilevel inheritance?

```
class Grandparent: pass
class Parent(Grandparent): pass
class Child(Parent): pass
```

19. What is polymorphism in Python? Explain with an example.
Answer: Same method behaves differently for different objects.
Example: Dog.speak() and Cat.speak().

20. Does Python support method overloading in the same way as languages like Java? Explain your answer.
Answer: No. Python uses default arguments, *args, and **kwargs instead.

21. How does method overriding work in Python? Give a code example.

```
class Parent:
    def show(self):
        print("Parent")

class Child(Parent):
    def show(self):
        print("Child")
```

22. What is the purpose of a module in Python?
Answer: Modules organize reusable code and functions.

23. How do you import a built-in module, such as 'math', and use its 'sqrt' function?

```
import math
print(math.sqrt(25))
```

24. Write the steps to create your own Python module and import it into another script.
Answer: 1. Create mymodule.py. 2. Add functions. 3. Import using: import mymodule.

25. What is the difference between a module and a package in Python?
Answer: • Module = Single .py file. • Package = Collection of modules in a directory.

26. What is the use of the 're' module in Python?
Answer: It provides regular expression functionality for pattern matching.

27. How does the match() function in the re module differ from search()?
Answer: • match() checks only at the beginning. • search() scans the entire string.

28. Write a Python code snippet to find all occurrences of the word 'python' in a text using re.findall().

```
import re
text = "python is great. python is easy."
print(re.findall("python", text))
```

29. What is Tkinter used for in Python?
Answer: Tkinter is Python's standard GUI library used for desktop applications.

30. How would you create a basic Tkinter window with a label that says 'Welcome'?

```
from tkinter import *
root = Tk()
Label(root, text="Welcome").pack()
root.mainloop()
```

31. Describe how you would add a button to a Tkinter window that prints 'Button clicked' when pressed.

```
from tkinter import *
def click():
    print("Button clicked")
root = Tk()
Button(root, text="Click", command=click).pack()
root.mainloop()
```

32. What are the common layout managers in Tkinter, and how do they differ?
Answer: • pack() = Automatic arrangement. • grid() = Row-column layout. • place() = Exact coordinates.

33. How do you connect a Python application to a MySQL database using pymysql?

```
import pymysql
conn = pymysql.connect(host="localhost", user="root", password="",
database="testdb")
```

34. Write a code snippet to insert a new record into a 'students' table using Python and pymysql.

```
import pymysql
conn = pymysql.connect(host="localhost", user="root", password="",
database="testdb")
cur = conn.cursor()
sql = "INSERT INTO students(name, age) VALUES (%s, %s)"
cur.execute(sql, ("Richa", 21))
conn.commit()
conn.close()
```

35. Explain how you would fetch all rows from a table in MySQL using Python.

```
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
```

```
for row in rows:
    print(row)
```

36. What precautions should you take when working with databases in Python to avoid SQL injection?
Answer: • Use parameterized queries. • Validate user input. • Avoid string concatenation in SQL statements.

37. How would you use both regular expressions and file handling together to extract all email addresses from a text file?

```
import re
with open("data.txt","r") as file:
    text = file.read()
emails = re.findall(r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
text)
print(emails)
```

38. If you have a class that manages database connections and you want to ensure it only creates one connection object, which OOP concept would you use and how?
Answer: Use the Singleton Design Pattern to ensure only one instance exists.

39. How could you use ChatGPT or GitHub Copilot to help you debug a Python script that is not working as expected, and what would you do to make sure the suggestions are safe to use?
Answer: Share the code and errors, review suggestions carefully, test in a safe environment, verify logic, and never blindly trust AI-generated code.